

Beatriz São Leandro Cosimatti  
17024130

João Victor Bozola Bosi  
5979703

João Vitor Vasconcelos Jacob  
16987258

Laís Antunes Cayres Quintão  
16880392

Trabalho de Estrutura de Dados II:  
**Análise e Implementação de Algoritmos**

Disciplina: Estrutura de Dados II  
SCC0606  
Professor: Careca safado

São Carlos  
14 de Junho de 2026

# 1 Introdução

O objetivo do trabalho desenvolvido foi estudar e implementar diferentes algoritmos de ordenação na linguagem C. Como cada algoritmo se comporta de uma maneira específica dependendo do tamanho do vetor ou de como os números estão organizados, o objetivo final do projeto é a criação de um Sistema Adaptativo. Esse sistema funciona como um analisador que avalia o vetor de entrada e, através de heurísticas, escolhe automaticamente o melhor algoritmo para realizar a ordenação.

Para fundamentar as tomadas de decisão desse sistema, o projeto foi estruturado de maneira modular, englobando a implementação de cinco algoritmos: Bubble Sort, Insertion Sort, Quick Sort, Heap Sort e Radix Sort.

ACHO Q COLOCA O LINK DO VIDEO AQ

## 2 Algoritmos Base

### 2.1 Bubble Sort

O Bubble Sort (Ordenação por Bolha) é um algoritmo de ordenação baseado no princípio de comparações e trocas adjacentes. Ele percorre o vetor repetidamente a partir do início comparando pares de elementos vizinhos (vetor[i] e vetor [i+1]), e troca-os de lugar se o elemento da esquerda for maior que o da direita. Esse processo é repetido até que nenhuma troca seja necessária em uma passada completa, indicando que o conjunto de dados está totalmente ordenado, sendo essa a versão aprimorada do Bubble Sort.

Dessa forma, a sua complexidade de tempo é  $O(n^2)$ , exceto no caso em que o vetor já está ordenado. Nessa situação, o vetor só é percorrido uma única vez, tornando a complexidade  $O(n)$ . É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes devido ao crescimento quadrático das operações de comparação e movimentação de dados.

### 2.2 Insertion Sort

O Insertion Sort (Ordenação por Inserção) consiste em inserir um novo elemento em um vetor já ordenado ou quase ordenado. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é inserida. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é finalmente inserida.

Assim, sua complexidade de tempo é  $O(n^2)$ , exceto no caso em que o vetor já está ordenado (ou quase ordenado). Nessa situação, a chave é comparada apenas uma vez com o elemento imediatamente anterior e o loop interno é interrompido por um break, tornando a complexidade  $O(n)$ . É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes e aleatórias.

### 2.3 Quick Sort

O algoritmo de ordenação Quick Sort foi desenvolvido e publicado pela primeira vez em 1961, pelo cientista da computação britânico Charles Antony Richard Hoare (1934 - 2026). A técnica de ordenação foi concebida enquanto o cientista estudava na Universidade de Moscou e trabalhava no *National Physical Laboratory (NPL)*, do Reino Unido. Na ocasião, Hoare estava participando de um projeto que tinha como objetivo a tradução de textos em russo para inglês e, para isso, como os dicionários eram armazenados em fita magnética, necessitava ordenar as palavras dentro de uma sentença em ordem alfabética. Nesse contexto, surgiu a necessidade de desenvolver um algoritmo de ordenação que fosse assintoticamente menor que  $O(n^2)$ .

Sob essa ótica, o algoritmo é baseado no paradigma da divisão e conquista. Nesse sentido, a aplicação se consiste em ordenar o vetor em torno de um pivô escolhido, de modo que, os elementos à esquerda do pivô sejam menores que ele e os elementos à direita, consequentemente, sejam maiores. Dessa forma, o vetor é subdividido em duas partições, que terão o mesmo processo aplicado sobre elas, através de chamadas recursivas da função. Assim, as chamadas se encerram quando o caso base de um elemento na partição é atingido, uma vez que ele já estará ordenado.

Tais comportamentos destacados são os aspectos base do algoritmo quick sort, no entanto, cada aplicação dista da técnica utilizada para escolher o elemento pivô e para como o particionamento será realizado. Para o algoritmo implementado neste projeto, utilizou-se o método da mediana de 3 para se determinar o pivô. Ele se consiste em armazenar o primeiro, central e último elemento do vetor em variáveis auxiliares e retornar a mediana entre eles, nesse caso, o elemento do meio. Essa técnica, embora seja mais complexa do que aplicações comuns que utilizam o primeiro ou último elemento do array como pivôs, tem como benefícios a melhora do tempo de execução e a possibilidade da execução possuir uma complexidade de  $O(n \log n)$  mesmo para vetores ordenados ou parcialmente ordenados - embora as métricas de análise dos vetores utilizadas para esse projeto evitem tais casos.

No caso da partição, existem três principais métodos comumente utilizados para o quick sort, o particionamento de Lomoto, Naive e Hoare, para este trabalho se utilizou o particionamento de Hoare. O método se consiste em na definição de dois ponteiros, nas duas extremidades do vetor e que, a cada interação, se movem em direção de seu encontro até que um elemento que necessita ser trocado de lugar seja encontrado. Considerando  $i$  o elemento na posição zero do vetor e  $j$  o elemento na posição final ambos são movidos até que  $i$  encontre um elemento maior que o pivô e, analogamente,  $j$  encontre um elemento menor que o pivô. Nesse caso, os valores encontrados trocam de posição e  $i$  e  $j$  voltam a percorrer o vetor até que eles se cruzem ou sobreponham. Destaca-se, que assim como a partição de Lomoto, o algoritmo de Hoare possui uma complexidade de tempo de  $O(n)$  e de espaço de  $O(1)$ , contudo, ela é considerada mais rápida que o outro método, uma vez que, no geral, realiza menos trocas entre elementos. Em relação ao particionamento de Naive, o algoritmo se baseia na criação de um vetor auxiliar, ou seja, a complexidade de espaço é de  $O(n)$ , sendo significativamente menos eficiente que o método utilizado.

Dado as características da partição de Hoare, é importante destacar que o quick sort é um algoritmo *in-place*, ou seja, não utiliza nenhum vetor auxiliar para a ordenação, uma vez que as trocas são realizadas dentro do próprio vetor. Além disso, o algoritmo não é estável, uma vez que não preserva a ordem relativa entre elementos de mesmo valor. É também importante destacar que o quick sort apresenta um comportamento muito eficiente em termos de acesso à memória devido à forma como realiza o particionamento do vetor. Durante essa etapa, os elementos são percorridos sequencialmente por meio de índices que avançam a partir das extremidades do arranjo, favorecendo a localidade espacial dos dados. Como elementos próximos na memória tendem a ser acessados em instantes próximos, a hierarquia de memória do processador, especialmente os níveis de cache, é utilizada de maneira mais eficiente. Além disso, à medida que as partições são realizadas, o algoritmo passa a operar sobre subvetores progressivamente menores, os quais frequentemente passam a caber integralmente na memória cache. Dessa forma, reduz-se a quantidade de acessos à memória principal, resultando em uma diminuição da latência de acesso aos dados e contribuindo significativamente para o elevado desempenho prático do algoritmo.

Ademais, é possível analisar o melhor, pior e médio caso do quick sort. Nota-se que o melhor caso ocorre quando o pivô selecionado é exatamente o elemento central do vetor analisado.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Pela análise da recursão, através do Teorema Mestre, obtém-se a complexidade de espaço para o melhor caso como  $O(n \log n)$ . Nesse sentido, para o médio caso, considera-se que o array é dividido em duas partes de tamanho  $K$ , sendo assim:

$$T(n) = T(n - K) + T(n)$$

Após a resolução da recorrência, também nota-se que a complexidade é de  $O(n \log n)$ . Nesse sentido, evidencia-se que no melhor e médio caso, o quick sort utiliza em média  $2 n \log n$  comparações entre elementos distintos do vetor para ordená-lo, e aproximadamente, um sexto desse número de trocas. Por fim, o pior caso ocorre quando o pivô escolhido é sempre o menor ou o maior elemento do subvetor analisado. Nessa situação, a cada etapa do algoritmo são geradas partições extremamente desbalanceadas, contendo 0 e  $n - 1$  elementos, o que resulta em uma árvore de recursão linear e leva a uma complexidade temporal de  $O(n^2)$ :

$$T(n) = T(n - 1) + n$$

Para o pior caso, observa-se que o algoritmo necessita realizar, em média,  $n^2/2$  comparações para ordenar o vetor, o que representa um aumento assintótico significativo em relação aos outros casos. Em relação à complexidade de espaço, cada chamada recursiva apresenta complexidade  $O(1)$  - devido a característica *in-place* do algoritmo -, no entanto, múltiplas chamadas ocorrem simultaneamente, modificando o comportamento da complexidade espacial conforme o caso de execução. Para o médio e melhor caso, a árvore de recursão gerada é balanceada e possui profundidade de  $O(\log n)$ , o que acarreta em uma complexidade de espaço de  $O(\log n)$ . Em contrapartida, devido ao comportamento linear da árvore no pior caso, sua altura é de  $O(n)$ , acarretando em uma complexidade assintótica de  $O(n)$ . Assim, foi possível elaborar a seguinte tabela comparativa:

| Varição     | Complexidade de tempo | Complexidade de espaço |
|-------------|-----------------------|------------------------|
| Melhor caso | $O(n \log n)$         | $O(\log n)$            |
| Caso médio  | $O(n \log n)$         | $O(\log n)$            |
| Pior caso   | $O(n^2)$              | $O(n)$                 |

Tabela 1: Complexidade de tempo e espaço do quick sort.

## 2.4 Heap Sort

O algoritmo de ordenação Heap Sort foi inventado em 1964 pelo cientista da computação J. W. J. Williams. A sua criação foi um marco na computação, pois introduziu simultaneamente a estrutura de dados homônima, o *Heap* (ou monte). A motivação original de Williams era desenvolver um algoritmo que fosse superior ao Quick Sort no pior caso, garantindo um limite assintótico restrito sem sacrificar a eficiência do uso de memória. Posteriormente, no mesmo ano, Robert Floyd refinou o algoritmo com uma função matemática capaz de construir o heap em tempo linear, consolidando a versão clássica estudada até hoje.

Sob essa ótica, o Heap Sort opera convertendo o vetor original em uma árvore binária quase completa, mapeada implicitamente no próprio arranjo sem a necessidade de ponteiros estruturais. Para o propósito de ordenação crescente, utiliza-se a variante *Max-Heap*, cuja propriedade fundamental determina que o valor de qualquer nó pai, localizado no índice  $i$ , deve ser obrigatoriamente maior ou igual aos valores de seus filhos, localizados nos índices  $2i + 1$  e  $2i + 2$ .

O processo de ordenação é dividido em duas etapas principais. A primeira etapa é a construção do heap (**Build-Max-Heap**), que reorganiza o vetor desordenado de baixo para cima. A segunda etapa consiste na extração repetida do elemento máximo. Como o maior valor garantidamente reside na raiz da árvore (índice 0), ele é trocado de lugar com o último elemento do vetor. O tamanho virtual do heap é então decrementado, e a função auxiliar **Max-Heapify** é acionada para rebaixar a nova raiz até a sua posição correta, restaurando a propriedade original do *Max-Heap*. Esse ciclo se repete sucessivamente até que todo o subvetor se esgote.

Matematicamente, a eficiência do algoritmo é garantida por sua estrutura de árvore. A etapa inicial de construção do heap realiza um número de operações proporcional à altura dos nós. A soma dessas operações ao longo de todos os níveis converge para uma série geométrica cuja soma assintótica é de tempo estritamente linear:

$$O\left(\sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{h+1}} h\right) = O(n)$$

Para a segunda etapa, a função **Max-Heapify** percorre a altura da árvore, custando  $O(\log_2 n)$  a cada uma das  $n - 1$  extrações executadas. Como consequência, o tempo total de extração domina a complexidade final da equação, resultando na seguinte relação:

$$T(n) = O(n) + O(n \log_2 n) = O(n \log_2 n)$$

Dessa forma, diferentemente do Quick Sort, o Heap Sort é blindado contra o pior caso, garantindo a complexidade de  $O(n \log_2 n)$  independentemente da distribuição inicial ou do grau de desordem dos dados. Ademais, por ser um algoritmo rigorosamente *in-place*, sua complexidade espacial é de  $O(1)$ , exigindo apenas o espaço de uma variável temporária para a realização das trocas.

No entanto, é fundamental destacar o comportamento arquitetural deste algoritmo frente à hierarquia de memória moderna. Apesar de sua elegância teórica, o Heap Sort possui uma péssima localidade espacial de referência. Durante a fase de restauração do heap, a navegação em direção aos filhos nos índices  $2i + 1$  e  $2i + 2$  gera saltos enormes e matematicamente fragmentados na memória. Quando o tamanho do vetor supera a capacidade da memória cache (níveis L2 ou L3) do processador, o algoritmo provoca uma alta taxa de *Cache Misses*, obrigando a CPU a paralisar seus ciclos de clock enquanto aguarda a lenta busca dos dados na memória RAM, o que degrada substancialmente seu desempenho prático em relação a concorrentes de acesso sequencial.

Assim, foi possível elaborar a seguinte tabela comparativa:

| Varição     | Complexidade de tempo | Complexidade de espaço |
|-------------|-----------------------|------------------------|
| Melhor caso | $O(n \log_2 n)$       | $O(1)$                 |
| Caso médio  | $O(n \log_2 n)$       | $O(1)$                 |
| Pior caso   | $O(n \log_2 n)$       | $O(1)$                 |

Tabela 2: Complexidade de tempo e espaço do heap sort.

## 2.5 Radix Sort

O Radix Sort (Ordenação por Raiz) é um dos algoritmos de ordenação mais antigos da história da computação, com raízes que remontam a 1887. Ele foi inicialmente desenvolvido pelo engenheiro estatístico Herman Hollerith para ser utilizado fisicamente nas máquinas de tabulação de cartões perfurados durante o censo dos Estados Unidos de 1890. Ao contrário dos algoritmos tradicionais focados em comparações diretas de grandeza, o Radix Sort quebra o limite inferior computacional estrito de  $\Omega(n \log_2 n)$ , operando puramente como um algoritmo não-comparativo.

O seu mecanismo de funcionamento fundamenta-se na decomposição e análise estrutural dos elementos dígito a dígito. A variante mais clássica e abordada neste contexto é a versão *Least Significant Digit* (LSD), que processa os números da direita para a esquerda, iniciando pela casa das unidades, progredindo para as dezenas, centenas, e assim sucessivamente. Para que essa progressão sequencial funcione e o vetor finalize perfeitamente ordenado, o Radix Sort exige, de maneira irrenunciável, o uso de um algoritmo de ordenação auxiliar interno que seja **estável** — isto é, que preserve incondicionalmente a ordem relativa de chaves originariamente idênticas. Tradicionalmente, o Counting Sort é o algoritmo selecionado para desempenhar essa função estrutural.

Nesse sentido, a aplicação se inicia varrendo o vetor de dados para identificar o elemento de maior valor, com o propósito de determinar a quantidade máxima de dígitos ou bits ( $d$ ) inerente àquele conjunto. Em seguida, um laço de repetição invoca o algoritmo de Counting Sort exatas  $d$  vezes. A cada interação do laço, os elementos são agrupados e reagrupados em partições virtuais correspondentes ao valor do dígito que está sendo focado no momento (frequentemente de 0 a 9, caso seja base decimal).

Para a análise de complexidade de tempo, considera-se  $n$  como o tamanho total do vetor,  $d$  como a quantidade máxima de dígitos do maior elemento e  $k$  como a base numérica adotada no sistema. Observa-se que cada rodada do Counting Sort exige um tempo proporcional de  $O(n + k)$  para realizar a distribuição e reordenação. Como esse processo mecânico é repetido rigorosamente  $d$  vezes, a complexidade final é dada por:

$$T(n) = O(d \cdot (n + k))$$

Quando a magnitude dos dados inseridos é limitada por padrões arquiteturais e a quantidade de dígitos ( $d$ ) é assumida como uma constante (fenômeno comum ao se ordenar inteiros nativos de 32 bits), o escopo da variável  $n$  domina majoritariamente a equação matemática, permitindo que o comportamento assintótico se simplifique para um modelo de tempo temporal estritamente linear, denotado por  $O(n)$ .

Entretanto, a elevada eficiência temporal do Radix Sort é penalizada através dos severos requisitos de hardware que ele impõe. O algoritmo não pertence à classe *in-place*. O uso intrínseco do Counting Sort demanda a inicialização de vetores de frequência com tamanho  $k$  e, de forma muito mais sensível, de um vetor auxiliar com alocação idêntica ao do vetor original ( $n$ ) para possibilitar o remanejamento estável dos elementos na memória.

Dessa forma, em casos de processamento massivo envolvendo vetores de magnitude bilionária, a complexidade espacial de  $O(n+k)$  consolida-se como uma barreira física primária. A alocação excessiva e concomitante de recursos pode exaurir a capacidade física da memória RAM do computador, induzindo o sistema operacional a iniciar processos de paginação diretamente no disco de armazenamento secundário (HD ou SSD), o que eleva a latência e dilapida a vantagem da complexidade algorítmica. Para além disso, vale destacar que o método convencional apresenta disfunções sistêmicas quando submetido a números negativos, necessitando de rotinas adicionais de *offsetting* (deslocamento numérico unificado) para evitar violações de acesso ou interrupções.

Assim, foi possível elaborar a seguinte tabela comparativa:

| <b>Variação</b> | <b>Complexidade de tempo</b> | <b>Complexidade de espaço</b> |
|-----------------|------------------------------|-------------------------------|
| Melhor caso     | $O(d \cdot (n + k))$         | $O(n + k)$                    |
| Caso médio      | $O(d \cdot (n + k))$         | $O(n + k)$                    |
| Pior caso       | $O(d \cdot (n + k))$         | $O(n + k)$                    |

Tabela 3: Complexidade de tempo e espaço do radix sort.

## 3 Arquitetura do Sistema

### 3.1 Execução da Aplicação

A aplicação foi desenvolvida para ser executada por meio da linha de comando, permitindo ao usuário configurar diferentes parâmetros relacionados à geração dos dados de entrada e ao algoritmo de ordenação utilizado. A interface foi projetada de forma a facilitar a realização de experimentos em larga escala, possibilitando a automatização dos testes e a coleta sistemática das métricas de desempenho.

A sintaxe geral de execução do programa é dada por:

```
./programa [opções]
```

Em que as opções disponíveis permitem selecionar o algoritmo de ordenação, definir a origem dos dados de entrada e especificar características da distribuição dos valores.

#### 3.1.1 Seleção do Algoritmo

O algoritmo de ordenação pode ser escolhido por meio da flag `--algoritmo`. Atualmente, a aplicação suporta os algoritmos Heap Sort, Quick Sort, Insertion Sort, Bubble Sort e Radix Sort.

Por exemplo, para executar o Quick Sort sobre um vetor gerado dinamicamente com 100.000 elementos:

```
./programa --algoritmo quick --tamanho 100000
```

Também é possível utilizar o modo automático:

```
./programa --algoritmo auto --tamanho 100000
```

Nesse caso, o sistema realiza uma caracterização prévia do vetor e seleciona automaticamente o algoritmo considerado mais adequado para aquela entrada.

#### 3.1.2 Geração Dinâmica de Vetores

A geração dinâmica é realizada através da flag `--tamanho`, que define a quantidade de elementos do vetor.

```
./programa --algoritmo quick --tamanho 1000000
```

Além do tamanho, o usuário pode especificar a distribuição dos dados utilizando a flag `--distribuicao`. Por exemplo:

```
./programa --algoritmo quick --tamanho 1000000 --distribuicao aleatorio
```

As distribuições atualmente disponíveis são:

- aleatorio
- quase\_ordenado
- crescente
- decrescente
- discrepante
- organ\_pipe
- intercalado

Por exemplo, para gerar um vetor utilizando a distribuição crescente:

```
./programa --algoritmo quick --tamanho 1000000 --distribuicao crescente
```

### 3.1.3 Controle do Grau de Desordem

Para vetores quase ordenados, o usuário pode especificar o percentual de elementos que sofrerão trocas aleatórias através da flag `--desordem`.

Por exemplo, para gerar um vetor com apenas 5% de desordem:

```
./programa --algoritmo quick --tamanho 1000000 --distribuicao quase_ordenado --desordem 5
```

Da mesma forma, pode-se aumentar o grau de desordem:

### 3.1.4 Leitura de Dados a Partir de Arquivos

A aplicação também permite utilizar vetores previamente armazenados em arquivos texto. Para isso, utiliza-se a flag `--input` seguida do caminho do arquivo.

```
./programa --algoritmo heap --input dados.txt
```

Assim, o programa realiza automaticamente a leitura dos valores contidos no arquivo, determina o tamanho da entrada e executa o algoritmo selecionado. Observa-se que o programa contabiliza todos os diferentes valores do arquivo como um único vetor.

### 3.1.5 Inserção Manual de Dados

Para testes rápidos e depuração, a aplicação permite a inserção direta dos elementos do vetor pela linha de comando utilizando a flag `--array`.

Por exemplo:

```
./programa --algoritmo quick --array 8 3 1 7 5 2 9 4
```

Nesse caso, o vetor utilizado pelo algoritmo será:

[8 3 1 7 5 2 9 4]

### 3.1.6 Saída do Programa

Ao final da execução, a aplicação apresenta no terminal uma tabela contendo as principais métricas coletadas durante a ordenação, incluindo: Além da exibição em tela, todos os resultados são armazenados automaticamente em um arquivo CSV localizado no diretório `output`, permitindo posterior análise estatística e construção de gráficos.

## 3.2 Distribuições dos Vetores Utilizados

Com o objetivo de avaliar o comportamento dos algoritmos de ordenação sob diferentes condições de entrada, foram implementadas diversas estratégias de geração de vetores. Cada distribuição busca representar cenários específicos, permitindo analisar tanto o desempenho e a eficiência dos algoritmos diante de diferentes padrões de organização dos dados.

### 3.2.1 Vetor Aleatório

Nesta distribuição, cada posição do vetor recebe um valor pseudoaleatório gerado independentemente das demais posições. Os valores são obtidos por meio de uma distribuição uniforme em um intervalo previamente definido.

Para um vetor de tamanho ( $n$ ), um exemplo de instância gerada é:

$A = [42 \ 17 \ 91 \ 3 \ 58 \ 24 \ \dots]$

Essa distribuição representa o caso mais comum em análises experimentais e é frequentemente utilizada como referência para comparação entre algoritmos de ordenação.

### 3.2.2 Vetor Quase Ordenado

A distribuição quase ordenada é construída inicialmente em ordem crescente. Em seguida, uma porcentagem previamente definida dos elementos tem suas posições trocadas aleatoriamente.

Para um vetor de tamanho ( $n$ ), um exemplo é:

$$A = [1 \ 2 \ 3 \ 4 \ 8 \ 6 \ 7 \ 5 \ 9 \ 10 \ \dots \ n]$$

Essa distribuição permite avaliar a capacidade dos algoritmos de explorar a existência de ordenação prévia nos dados.

### 3.2.3 Vetor Discrepante

A distribuição discrepante é composta majoritariamente por valores pertencentes a um intervalo reduzido. Posteriormente, uma pequena fração dos elementos é substituída por valores extremamente elevados, denominados *outliers*.

Para um vetor de tamanho ( $n$ ), um exemplo é:

$$A = [15 \ 23 \ 8 \ 41 \ 10^9 \ 12 \ 4 \ 37 \ \dots]$$

Essa configuração permite analisar o impacto da presença de valores discrepantes no desempenho dos algoritmos de ordenação.

### 3.2.4 Vetor Crescente

Nesta distribuição, os elementos são armazenados em ordem estritamente crescente.

Para um vetor de tamanho ( $n$ ), a sequência gerada é dada por:

$$A = [1 \ 2 \ 3 \ \dots \ n-2 \ n-1 \ n]$$

Trata-se de um caso clássico utilizado na análise de algoritmos de ordenação, uma vez que determinadas estratégias de seleção podem apresentar comportamentos significativamente distintos quando os dados já se encontram ordenados.

### 3.2.5 Vetor Decrescente

Nesta distribuição, os elementos são armazenados em ordem estritamente decrescente.

Para um vetor de tamanho ( $n$ ), a sequência gerada é dada por:

$$A = [n \ n-1 \ n-2 \ \dots \ 3 \ 2 \ 1]$$

Assim como o vetor crescente, essa distribuição é amplamente utilizada na literatura para avaliar a sensibilidade dos algoritmos à disposição inicial dos dados.

### 3.2.6 Distribuição Organ Pipe

A distribuição *organ pipe* apresenta valores crescentes até uma posição central e, a partir desse ponto, valores decrescentes, formando uma estrutura semelhante aos tubos de um órgão musical, o que origina o seu nome.

De forma geral, para um vetor de tamanho ímpar ( $n$ ), a sequência possui a forma:

$$A = \left[ 1 \ 2 \ 3 \ \dots \ \frac{n+1}{2} \ \dots \ 3 \ 2 \ 1 \right]$$

Por exemplo, para ( $n=9$ ):

$$A = [1 \ 2 \ 3 \ 4 \ 5 \ 4 \ 3 \ 2 \ 1]$$

### 3.2.7 Distribuição Intercalada

A distribuição intercalada alterna elementos de pequeno e grande valor ao longo de todo o vetor. De forma geral, para um vetor de tamanho ( $n$ ), a sequência é construída segundo o padrão:

$$A = [1 \ n \ 2 \ n - 1 \ 3 \ n - 2 \ \dots]$$

Por exemplo, para ( $n=10$ ):

$$A = [1 \ 10 \ 2 \ 9 \ 3 \ 8 \ 4 \ 7 \ 5 \ 6]$$

Observa-se que as distribuições *organ pipe* e intercalada foram incluídas especificamente para investigar possíveis degradações de desempenho do quick sort utilizando a estratégia da mediana de três para seleção do pivô, que será especificado na devida sessão de resultados.

## 3.3 Dados de Desempenho Coletados

Diversos dados foram coletados durante as execuções e testes dos algoritmos, com o intuito de estudar sistematicamente o comportamento de cada método no cenários analisados. Nesse sentido, os dados de desempenho coletados foram:

- **Tempo de execução:** determinação do tempo de execução da ordenação em segundos;
- **Número de comparações:** número total de comparações entre elementos do vetor durante a execução;
- **Número de trocas:** mede a quantidade de trocas de valores entre dois elementos do vetor, nos códigos implementados, a função de troca foi modularizada e chamada por todos os algoritmos conforme a necessidade;
- **Número de operações:** quantifica a quantidade de operações totais realizada pelo algoritmo durante a execução - como atribuição de valores, operações matemáticas, comparações e outros-;
- **Número de cópias:** determina a quantidade de vezes que cópias de elementos e vetores são realizadas;
- **Chamadas recursivas:** quantifica o total de chamadas recursivas necessárias para a ordenação;
- **Profundidade Máxima da Recursão:** mede a altura máxima da árvore de recursão percorrida pelo algoritmo, ou de forma equivalente, O maior número de chamadas da função recursiva simultaneamente ativas na pilha de execução durante toda a ordenação;
- **Memória Extra Utilizada:** determina a quantidade de memória auxiliar utilizada para a ordenação através de uma estimativa envolvendo as variáveis auxiliares criadas e o tamanho delas em bytes.

Observa-se que nem todos os dados foram coletados para todos os algoritmos, uma vez que eles diferem da arquitetura de implementação - iterativo ou recursivo - e de particularidades mais intrínsecas. Dessa forma, algumas métricas podem não condizer com alguns algoritmos.

## 3.4 Métricas Coletadas

O sistema adaptativo desenvolvido tem como objetivo selecionar automaticamente o algoritmo de ordenação mais adequado para cada vetor de entrada. Para isso, antes da ordenação, são coletadas diversas métricas que descrevem as características dos dados. A partir dessas informações, um conjunto de heurísticas decide qual algoritmo apresenta maior potencial de desempenho para a entrada analisada.

### 3.4.1 Grau de Desordem

O grau de desordem mede o percentual de inversões adjacentes existentes no vetor. Uma inversão adjacente ocorre quando um elemento é maior que seu sucessor imediato.

A métrica é calculada por:

$$Desordem = \frac{\text{Quantidade de Inversões Adjacentes}}{n - 1} \times 100$$

Essa métrica foi utilizada para identificar vetores quase ordenados, situação na qual o Insertion Sort apresenta desempenho próximo de  $O(n)$ .

### 3.4.2 Quantidade de Elementos Repetidos

Esta métrica contabiliza quantos elementos do vetor são cópias extras de valores já existentes.

Seu objetivo inicial era investigar se a presença de muitos valores repetidos poderia influenciar a escolha do algoritmo de ordenação.

### 3.4.3 Amplitude dos Valores (Range)

A amplitude dos valores corresponde à diferença entre o maior e o menor elemento do vetor:

$$Range = Max - Min$$

Essa métrica foi utilizada para avaliar a adequação do vetor à aplicação do Radix Sort.

### 3.4.4 Desvio Padrão

O desvio padrão mede o grau de dispersão dos elementos em relação à média do conjunto de dados.

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

Inicialmente investigou-se a possibilidade de utilizar essa métrica como parâmetro para seleção do algoritmo.

### 3.4.5 Valores Máximo e Mínimo

Além de serem utilizados para o cálculo da amplitude dos dados, os valores máximo e mínimo permitem identificar a presença de números negativos, informação importante para a escolha do algoritmo de ordenação.

## 3.5 Seleção das Métricas

Embora todas as métricas tenham sido implementadas e avaliadas experimentalmente, apenas aquelas que demonstraram influência consistente no desempenho dos algoritmos foram incorporadas à heurística final do sistema.

| Métrica                 | Utilizada na heurística |
|-------------------------|-------------------------|
| Grau de desordem        | Sim                     |
| Quantidade de repetidos | Não                     |
| Amplitude (Range)       | Sim                     |
| Desvio padrão           | Não                     |
| Valor mínimo            | Sim                     |

Tabela 4: Métricas implementadas e utilização na heurística final

A quantidade de elementos repetidos e o desvio padrão foram mantidos apenas para fins de caracterização dos conjuntos de dados. Durante os experimentos realizados, essas métricas não apresentaram influência significativa na escolha do algoritmo mais eficiente.

### 3.6 Heurística Final

Após a realização dos experimentos, a seguinte estratégia foi adotada:

```
if (n <= 1000 && desordem <= 5.0)
    return "insertion";

if (min < 0 && n >= 10000)
    return "heap";

if (min >= 0 && range <= 10*n)
    return "radix";

return "quick";
```

A lógica empregada pode ser resumida da seguinte forma:

- Vetores pequenos e quase ordenados são processados pelo Insertion Sort.
- Vetores contendo valores negativos podem ser direcionados ao Heap Sort.
- Vetores cuja amplitude seja proporcional ao tamanho são direcionados ao Radix Sort.
- Os demais casos são tratados pelo Quick Sort.

A regra associada à presença de valores negativos foi incluída por razões teóricas relacionadas à aplicabilidade do Radix Sort.

Os conjuntos de dados utilizados durante os experimentos continham apenas valores não negativos, impossibilitando a validação experimental dessa decisão. Dessa forma, a utilização do Heap Sort para vetores negativos de grande porte foi baseada em propriedades teóricas dos algoritmos analisados e deve ser entendida como uma decisão conservadora da heurística.

## 4 Validação das Heurísticas

A definição da heurística final foi baseada em uma série de hipóteses experimentais avaliadas durante o desenvolvimento do projeto.

### 4.1 Hipótese 1: Vetores Pequenos Favorecem o Insertion Sort

Inicialmente assumiu-se que vetores com menos de 50 elementos deveriam ser processados pelo Insertion Sort.

Para validar essa hipótese, foram executados experimentos utilizando vetores aleatórios de tamanho 50 e comparando os resultados obtidos por todos os algoritmos implementados.

Os resultados mostraram que o Quick Sort apresentou desempenho superior ao Insertion Sort mesmo para esse tamanho de entrada.

Dessa forma, a regra:

```
if (n <= 50)
    return "insertion";
```

foi removida da heurística final.

**Conclusão: hipótese rejeitada.**

### 4.2 Hipótese 2: Vetores Quase Ordenados Favorecem o Insertion Sort

Foram realizados experimentos com vetores quase ordenados contendo níveis crescentes de desordem.

Observou-se que o Insertion Sort apresentou desempenho significativamente superior para vetores com grau de desordem reduzido, mantendo comportamento próximo ao caso linear esperado pela teoria.

A heurística baseada em:

```
n <= 1000 && desordem <= 5
```

mostrou-se consistente durante todos os experimentos realizados.

**Conclusão: hipótese confirmada.**

É importante destacar que vetores ordenados ou quase invertidos normalmente representam entradas problemáticas para implementações simples do Quick Sort que utilizam o primeiro ou o último elemento como pivô. Entretanto, a implementação desenvolvida neste trabalho utiliza a estratégia da mediana de três para seleção do pivô, considerando os elementos inicial, central e final do subvetor.

Essa abordagem reduz significativamente a probabilidade de partições extremamente desbalanceadas em vetores parcialmente ordenados ou invertidos. Como consequência, observou-se experimentalmente que o Quick Sort manteve desempenho compatível com o comportamento esperado de  $O(n \log n)$  nesses cenários.

### 4.3 Hipótese 3: O Radix Sort Necessita de um Tamanho Mínimo

Inicialmente foi considerada a utilização de um limite inferior para ativação do Radix Sort.

Entretanto, testes realizados com vetores de aproximadamente 5000 elementos mostraram que o algoritmo já apresentava desempenho superior ao Quick Sort.

Dessa forma, o limite mínimo inicialmente considerado foi removido.

**Conclusão: hipótese rejeitada.**

### 4.4 Hipótese 4: Vetores Grandes Favorecem o Radix Sort

Experimentos realizados com vetores aleatórios de 10000 elementos confirmaram que o Radix Sort apresenta desempenho superior quando a amplitude dos valores é compatível com o tamanho do vetor.

**Conclusão: hipótese confirmada.**

## 4.5 Hipótese 5: Vetores Discrepantes Devem Evitar o Radix Sort

Considerou-se inicialmente que vetores com amplitudes extremamente elevadas deveriam evitar o uso do Radix Sort.

Os resultados mostraram que, mesmo em cenários discrepantes, o Radix Sort continuou apresentando desempenho competitivo. Entretanto, observou-se que o aumento da amplitude dos dados pode tornar o Quick Sort uma alternativa interessante devido ao menor consumo de memória auxiliar.

Dessa forma, conclui-se que a amplitude dos dados deve ser considerada pela heurística, porém não foi observada uma degradação suficientemente significativa que justificasse a exclusão completa do Radix Sort nesses cenários.

**Conclusão: hipótese parcialmente aceita.**

## 4.6 Hipótese 6: Vetores Muito Grandes Favorecem o Heap Sort

Também foi investigada a possibilidade de utilizar o Heap Sort para vetores extremamente grandes. Foram realizados experimentos com vetores contendo até 500000 elementos.

Os resultados mostraram que o Heap Sort permaneceu consistentemente inferior ao Quick Sort e ao Radix Sort em termos de tempo de execução e número total de operações.

Consequentemente, não foi identificada uma situação experimental em que sua utilização apresentasse vantagens claras.

**Conclusão: hipótese rejeitada.**

## 5 Metodologia Experimental

Descrever: • ambiente utilizado; • datasets; • tamanhos testados; • critérios de avaliação

## 6 Resultados

Apresentar tabelas, gráficos, métricas coletadas, com discussão crítica. Explicar: • decisões corretas; • decisões ruins; • comportamentos inesperados; • limitações observadas; • impacto das características da entrada. • impactos das entradas adversariais

### 6.1 Bubble Sort

Ao implementar o Bubble Sort, foi utilizada uma flag booleana (houveTroca) para identificar se o vetor está ordenado após percorrê-lo. Assim, essa estratégia corroborou a eficiência do algoritmo diante de entradas já ordenadas. Diante de diversas entradas com diferentes características, foi possível organizar as informações na Tabela 5.

| Tamanho (n) | Cenário de entrada        | Tempo de execução | Comparações | Trocas |
|-------------|---------------------------|-------------------|-------------|--------|
| 1000        | Já Ordenado (Melhor caso) | a                 | a           | 0      |
| 1000        | Aleatório                 | a                 | a           | a      |
| 1000        | Invertido (Pior caso)     | a                 | a           | a      |

Tabela 5: Tabelas e Métricas coletadas - BubbleSort.

A fim de visualizar melhor a complexidade temporal do Bubble Sort nos três casos distintos (melhor caso, caso médio e pior caso), foram construídos gráficos em função do tamanho N do vetor e do tempo de execução.

#### GRAFICO MELHOR CASO

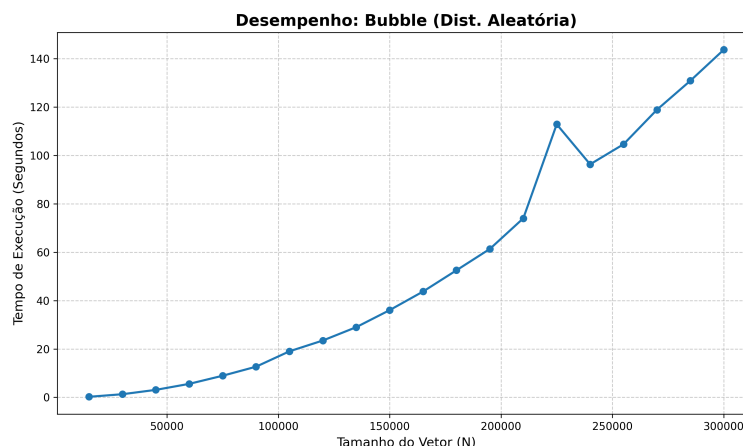


Figura 1: Desempenho Bubble Sort: Caso médio

#### GRAFICO PIOR CASO

Analisando os três gráficos gerados, observa-se que o algoritmo se comporta da maneira esperada para as entradas recebidas, possuindo uma complexidade temporal quadrática  $O(n^2)$  no caso médio (entradas aleatórias) e pior caso, e complexidade temporal  $O(n)$  no melhor caso (vetor ordenado). No gráfico da Figura NUMERO, que exibe o comportamento no caso médio, é possível observar um pico inesperado no tempo de execução para o vetor de tamanho 225.000. Como o número de comparações e trocas matemáticas manteve o crescimento estritamente quadrático correspondente à teoria assintótica, conclui-se que essa variação não possui relação com a lógica interna do Bubble Sort, e sim de uma interferência ambiental externa relacionada ao gerenciamento de processos do Sistema Operacional. Processos que rodam em segundo plano disputam o uso do processador, diminuindo temporariamente o tempo de execução dedicado ao programa.

A respeito do impacto das características da entrada, nota-se que o comportamento do Bubble Sort é fortemente governado pelo estado inicial do vetor. O algoritmo responde de forma excelente a vetores já ordenados devido à otimização por flag implementada, reduzindo o custo computacional de quadrático para linear. Contudo, em cenários onde a desordem é moderada ou total (aleatório),

o algoritmo perde eficiência rapidamente, pois o número de passadas necessárias para arrastar os elementos até suas posições corretas cresce massivamente.

Nesse tipo de algoritmo, a entrada adversarial para o Bubble Sort ocorre quando o vetor está perfeitamente invertido (ordem decrescente). Neste cenário, o elemento que deveria estar na última posição está na primeira, forçando o algoritmo a realizar o limite máximo teórico de comparações e trocas através da fórmula  $\frac{n(n-1)}{2}$ . Para grandes volumes de dados ( $n > 10.000$ ), essa quantidade de operações sobrecarrega o processador, tornando o algoritmo lento demais para ser utilizado.

## 6.2 Quick Sort

Com o intuito de validar a algoritmo quick sort implementado neste projeto, analisou-se as curvas do número de comparações entre elementos do vetor e número de trocas necessárias para atingir a ordenação - para valores com diferentes ordens de grandeza - quais foram estudadas teoricamente e possuem comportamento esperado definido. Dessa forma, nota-se que o número de comparações do quick sort para o melhor caso é na ordem de  $2 n \log n$ , e um sexto desse número para a quantidade de trocas.

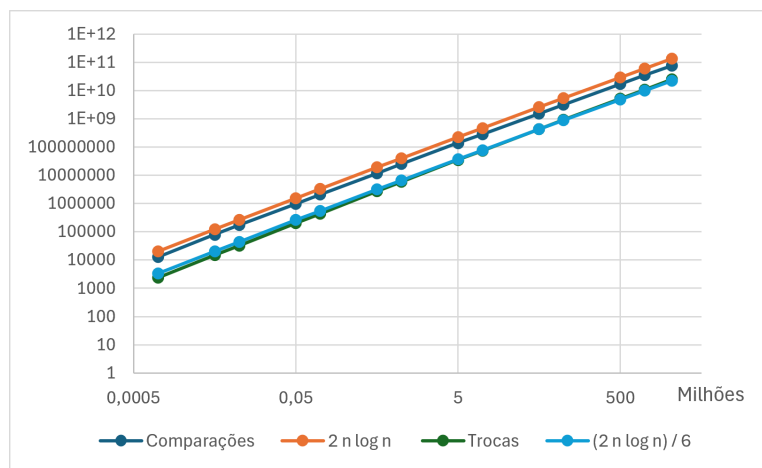


Figura 2: Comparativo entre o número de comparações e trocas e seu comportamento esperado para o melhor caso do quick sort. Escala dílog.

No gráfico é possível observar que o comportamento esperado é efetivo, uma vez que os dados coletados acompanham as linhas das funções que descrevem o comportamento previsto. Observa-se que o gráfico foi elaborado em escala log-log para a melhor visualização do comportamento para as diferentes ordens de grandeza.

Na implementação do quick sort desenvolvida, o pivô é escolhido por meio do método da mediana de três, utilizando os elementos inicial, central e final do subvetor. Essa estratégia reduz significativamente a ocorrência de partições desbalanceadas, especialmente em vetores já ordenados ou parcialmente ordenados. Ainda assim, o pior caso permanece teoricamente possível e ocorre quando a mediana desses três elementos produz repetidamente pivôs próximos aos extremos do subvetor, gerando partições de tamanhos aproximadamente 0 e  $n - 1$ . Nesse sentido, o método da mediana de três foi amplamente estudada por cientistas como David Musser, pesquisador conhecido por desenvolver o algoritmo Introsort, atualmente utilizado em diversas bibliotecas padrão da linguagem C++. Em seu trabalho, Musser demonstrou que, embora a estratégia da mediana de três reduza significativamente a probabilidade de partições desbalanceadas em entradas comuns, ela não elimina completamente o pior caso do Quick Sort. Sob essa ótica, existem famílias de sequências especialmente construídas, conhecidas como *Median-of-Three Killer Sequences*, capazes de induzir a escolha repetida de pivôs inadequados, resultando em partições altamente desbalanceadas e levando o algoritmo a uma complexidade temporal quadrática  $O(n^2)$ .

Algumas dessas sequências conhecidas são a *organ pipe* e intercalada. A sequência *organ pipe* se consiste em um array com o elemento central possuindo o maior valor do vetor e os elementos a esquerda e a direita dele decrescendo. Um exemplo de uma distribuição dessa forma, contínua, para

$n = 10$  é:  $[1\ 2\ 3\ 4\ 5\ 6\ 4\ 3\ 2\ 1]$ . Em relação a distribuição intercalada, o vetor se consiste em valores intercalados entre uma sequência crescente e outra decrescente, dessa forma, para  $n = 10$ , uma possível configuração seria  $[1\ 10\ 2\ 9\ 3\ 8\ 4\ 7\ 5\ 6]$ . No gráfico a seguir, é possível visualizar um comparativo entre os tempos de execução para vetores de distribuição aleatória e distribuição seguindo o padrão *organ pipe*:

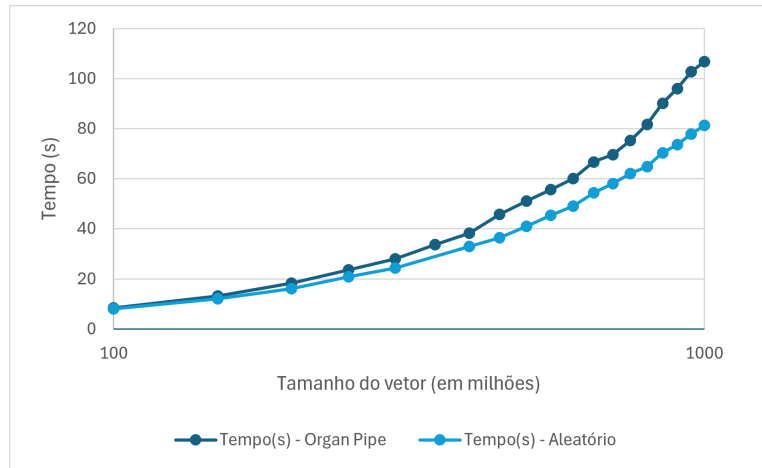


Figura 3: Comparatio de tempo de execução para a ordenação quick sort entre vetores de distribuição aleatória e quick sort.

Através dos dados, é possível observar que essa distribuição não estressou devidamente o algoritmo, uma vez que a curva de ambos apresentou uma complexidade temporal de  $O(n \log n)$ , embora a dsitribuição *organ pipe* tenha apresentado tempos de execução ligeiramente maiores. É válido destacar o comportamento da métrica que mede a profundidade máxima árvore de rescursão para ambos os casos.

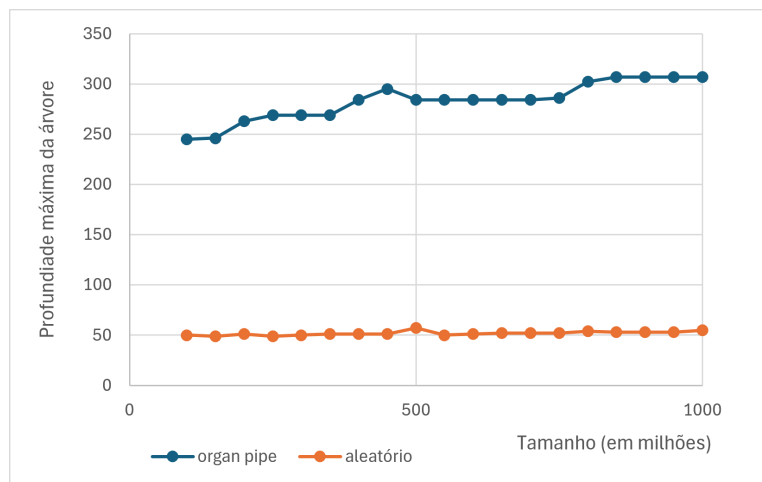


Figura 4: Comparatio de tempo de execução para a ordenação quick sort entre vetores de distribuição aleatória e quick sort.

Observa-se que a profundidade máxima para a distribuição *organ pipe* foi significativamente maior do que para o caso aleatório, indicando que a escolha do pivô para esses vetores não foi tão eficaz quanto o observado no melhor caso, o que acarretou na necessidade de um maior número de partições e, consequentemente, uma maior árvore recursiva. Assim, os arrays criandos seguindo o modelo *organ pipe* não estressaram o algoritmo implementado o suficiente para prejudicar a sua complexidade assintótica de tempo para  $O(n^2)$ , à medida que, como observado e analisado, a ditribuição se enquadra em um médio caso do quick sort.

Em relação aos vetores intercalados, o quick sort não conseguiu ordenar arrays com mais de 100 milhões de tamanho, uma vez que a péssima escolha dos pivôs para esse caso acarretaram em árvores de recursão extremamente grandes - com profundidade máxima maior que 80.000 para vetores na ordem de 200 milhões de tamanho - o que ocasionou a interrupção prematura da ordenação devido ao esgotamento da pilha de chamadas. Dessa forma, observa-se que o pior caso esperado foi atingido, acarretando em uma execução na ordem de  $O(n^2)$  e incapaz de ser finalizada para vetores grandes, como os analisados. Nesse sentido, evidencia-se também que para vetores menores aos analisados, de ordem de grandeza 10.000, a profundidade máxima da recursão atingiu valores muito maiores do que a ordenação de vetores aleatórios na casa de 100 milhões de tamanho - por exemplo, um vetor aleatório de tamanho 1 bilhão atingiu profundidade máxima de 55, enquanto um vetor intercalado de 100 mil, atingiu 507. Esses comportamentos destacam que a estratégia da mediana de três, embora eficiente em casos usuais, permanece suscetível a entradas adversárias capazes de produzir partições fortemente desbalanceadas.

## 7 Reflexão sobre Uso de IA

O grupo deverá informar:

- quais ferramentas de IA foram utilizadas;
- em quais etapas auxiliaram;
- erros encontrados nas respostas geradas;
- correções realizadas pelo grupo.

## 8 Conclusão

Resumo dos principais achados.

## 9 Referências Bibliográficas